

Arduino for Musicians

Alex Grant

Shameless Plugs

Push/Pull - Deeply Hanging Out

12:00 - 20:30 Saturday 24 May

Adelaide Contemporary Experimental

Durational performance

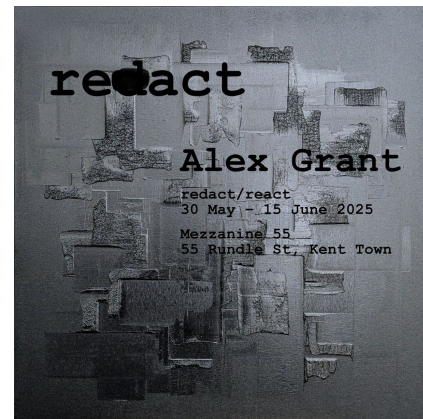


redact/react

30 May - 15 June

Mezzanine 55

Solo exhibition



Opening 6:00 - 20:00 Friday 30 May

Artist talk 2pm Saturday 7 June

Workshop Plan

Week 1 [Tues, May 13th]

Introduction to Arduino: Focusing on the Arduino microcontroller, participants will be introduced to basics of electronics.



Week 2 [Tues, May 20th]

MIDI: An introduction to the MIDI standard for interconnecting music equipment. Participants will learn how MIDI sends messages for notes and control values. We will build a simple system using the Arduino to send MIDI commands to a digital audio workstation.

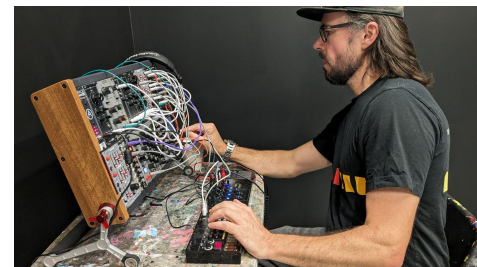


Week 3 [Tues, May 27th]

Modular Synthesis: Using VCV Rack participants will explore sound design and music production using modular synthesis.

Week 4 [Tues, June 3rd]

Homebrew MIDI Controllers: Bringing together the previous three workshops, in this session, participants will build custom MIDI controllers using Arduino and various sensors.



Prerequisites

Week 1 [Tues, May 13th]

- Laptop with [Arduino IDE](#) installed
- Optional: [fritzing](#)
- USB-A to USB-C converter or hub if needed
- Arduino starter kit (provided)



Week 2 [Tues, May 20th]

- Laptop with [Ableton Live](#) and [MIDIView](#) installed
- Alternatively [Garageband](#), [Tracktion](#) or whatever DAW
- Headphones
- Arduino starter kit



Week 3 [Tues, May 27th]

- Laptop with [VCV Rack](#) Installed
- Headphones



Week 4 [Tues, June 3rd]

- Laptop with Arduino IDE, Ableton Live, VCV Rack and MIDIView installed
- Headphones
- Arduino starter kit



Lab Guest
TheLab2024?

Download the Code



<https://github.com/grantaj/labrats-arduino>

git is a tool for tracking changes in files

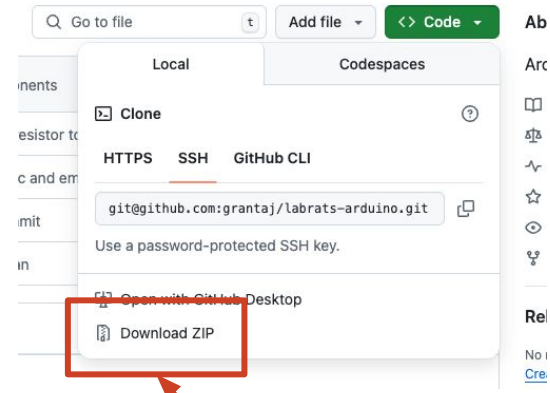
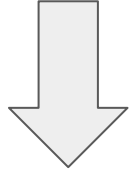
Saves versions of work so you can go back to earlier stages

You can work on different features or ideas in parallel using "branches"

Git works locally, but you can also use websites like **GitHub** to share your work and collaborate

Helps everyone stay organized and avoid overwriting each other's changes

Every change you save with Git is called a "commit", each commit has a message describing the change.



download the code as a .zip archive to your computer

Introduction to MIDI

Outline

What is MIDI?

5-Pin DIN MIDI

USB MIDI

Sending Notes

Sending pitch bend

Basic MIDI sequencer

Fancy Sequencer

What is MIDI?

- MIDI stands for **Musical Instrument Digital Interface** – invented in the 1980s.
- MIDI sends **performance data**, not sound – it tells instruments *what* to play, not *how* it sounds.
- Used to **connect instruments, computers, and controllers** – like keyboards, drum pads, synths, and DAWs.
- **Common MIDI messages** include:
 - *Note On/Off* – plays and stops a note
 - *Control Change (CC)* – changes things like volume, filters, or modulation
 - *Pitch Bend* – smoothly raises or lowers pitch
- **Standard MIDI values range from 0 to 127** – like note numbers and controller values.
- **MIDI can travel over different connectors** – 5-pin DIN, USB, or 3.5mm TRS cables.



Tip-Ring-Sleeve (TRS) MIDI

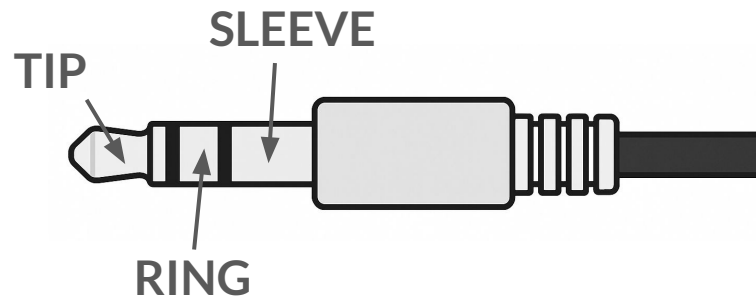
Type A

- **Tip:** MIDI Data (TX)
- **Ring:** Ground
- **Sleeve:** +5V
- Officially adopted by the **MIDI Manufacturers Association (MMA)**.



Type B

- **Tip:** +5V
- **Ring:** MIDI Data (TX)
- **Sleeve:** Ground



IN/OUT/THRU

IN - listens to MIDI data to control the instrument

OUT - sends MIDI data to control another instrument

THRU - retransmits an exact copy of all incoming MIDI data to OUT



MIDI Serial Data Format

- **Baud rate:** 31,250 bits per second (bps)
- **Data format:** 8-N-1
 - 8 data bits
 - No parity bit
 - 1 stop bit (high)
 - 1 start bit (low)
- **Electrically:**
 - Bits sent using *current* (more robust to noise)
 - Current flowing: Logic 0
 - Current off: Logic 1
 - Isolated using optocoupler on MIDI in (avoid ground loops)

| Start (0) | D0 | D1 | D2 | D3 | D4 | D5 | D6 | D7 | Stop (1) |

Binary

Binary represents numbers using only **0** and **1** (easy for electronics - off or on)

Each digit (called a **bit**) represents a power of 2 (instead of a power of 10)

13 = 0b1101

Decimal	
10s	1s
1	3
1x10	3x1
10+3 = 13	

Binary			
8s	4s	2s	1s
1	1	0	1
1x8	1x4	0x2	1x1
8+4+1 = 13			

Hexadecimal

Hexadecimal represents numbers using 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

Each digit represents a power of 16 (instead of a power of 10)

$$123 = 0x7B$$

Decimal			Hexadecimal		
100s	10s	1s	256s	16s	1s
1	2	3	0	7	B
1x100	2x10	3x1	0*256	7x16	11x1
100+20+3 = 123			112+11 = 123		

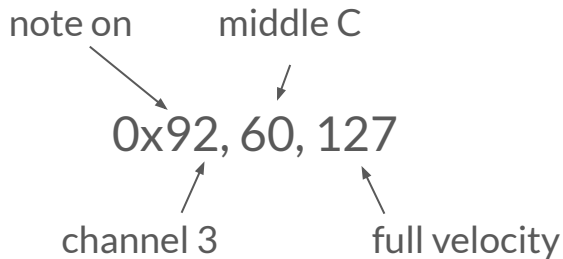
MIDI Note Messages

Messages are sent on one of 16 MIDI channels (0x0 - 0xF)

MIDI Channel 1 is represented as 0x0, Channel 16 is 0xF (numbering starts from zero)

Note On: 0x90–0x9F, followed by note number (0–127) and velocity (0–127).

Note Off: 0x80–0x8F, followed by note number and velocity.



Octave	C	C#/D ♭	D	D#/E ♭	E	F	F#/G ♭	G	G#/A ♭	A	A#/B ♭	B
-2	0	1	2	3	4	5	6	7	8	9	10	11
-1	12	13	14	15	16	17	18	19	20	21	22	23
0	24	25	26	27	28	29	30	31	32	33	34	35
1	36	37	38	39	40	41	42	43	44	45	46	47
2	48	49	50	51	52	53	54	55	56	57	58	59
3	60	61	62	63	64	65	66	67	68	69	70	71
4	72	73	74	75	76	77	78	79	80	81	82	83
5	84	85	86	87	88	89	90	91	92	93	94	95
6	96	97	98	99	100	101	102	103	104	105	106	107
7	108	109	110	111	112	113	114	115	116	117	118	119
8	120	121	122	123	124	125	126	127				

MIDI Control Messages

Control Change (CC): Adjusts a parameter like volume or filter.

0xB0–0xBF, followed by controller number (0–127) and value (0–127).

Program Change: Switches instrument sound (e.g. piano to strings).

0xC0–0xCF, followed by program number (0–127).

Pitch Bend: Raises or lowers the pitch.

0xE0–0xEF, followed by a 14-bit value as 2 data bytes (LSB + MSB).

CC #	Controller Name	Typical Use
0	Bank Select	Change sound banks
1	Modulation Wheel	Vibrato or modulation depth
2	Breath Controller	Wind instrument expression
4	Foot Controller	Foot pedal control
5	Portamento Time	Glide time between notes
6	Data Entry	Input for RPN/NRPN values
7	Channel Volume	Master volume
8	Balance	Stereo balance
10	Pan	Left-right stereo placement
11	Expression	Volume nuance or secondary volume

CC #	Controller Name	Typical Use
64	Sustain Pedal	Hold notes (on ≥ 64 , off < 64)
65	Portamento On/Off	Glide between notes
66	Sostenuto Pedal	Sustain only held notes
67	Soft Pedal	Softer tone (piano-style)
68	Legato Footswitch	Legato phrasing

Higher Resolution CC Values

- MIDI originally used 7-bits for control change values (0–127).
- To get finer resolution (0–16,382), combine two CC messages:
 - MSB: The main (most significant) 7-bit value, sent on CC
 - LSB: Optional (least significant) 7-bit value, sent on CC + 32

Example:

- CC 1 Modulation Wheel (MSB)
- CC 33 Modulation Wheel Fine (LSB)

The Official List of MIDI Messages

<https://midi.org/expanded-midi-1-0-messages-list>

Demo



MidiView

ENABLE LAUNCH CONTROL XL ABOUT CLEAR

Time	Dir	Message	Channel	Value	Hex
04:16:50.647	out	Controller Sound Control 8: 87	9	87	B8 4D 57
04:16:50.647	out	Controller Sound Control 8: 86	9	86	B8 4D 56
04:16:50.647	out	Controller Sound Control 8: 85	9	85	B8 4D 55
04:16:50.647	out	Controller Sound Control 8: 84	9	84	B8 4D 54
04:16:50.647	out	Controller Sound Control 8: 83	9	83	B8 4D 53
04:16:50.647	out	Controller Sound Control 8: 82	9	82	B8 4D 52
04:16:50.647	out	Controller Sound Control 8: 81	9	81	B8 4D 51
04:16:50.647	out	Controller Sound Control 8: 80	9	80	B8 4D 50
04:16:50.647	out	Controller Sound Control 8: 79	9	79	B8 4D 4F
04:16:50.647	out	Controller Sound Control 8: 78	9	78	B8 4D 4E
04:16:50.647	out	Controller Sound Control 8: 77	9	77	B8 4D 4D
04:16:50.647	out	Controller Sound Control 8: 76	9	76	B8 4D 4C
04:16:50.647	out	Controller Sound Control 8: 75	9	75	B8 4D 4B
04:16:51.647	out	Controller Sound Control 8: 76	9	76	B8 4D 4C



Outline

What is MIDI?

5-Pin DIN MIDI

USB MIDI

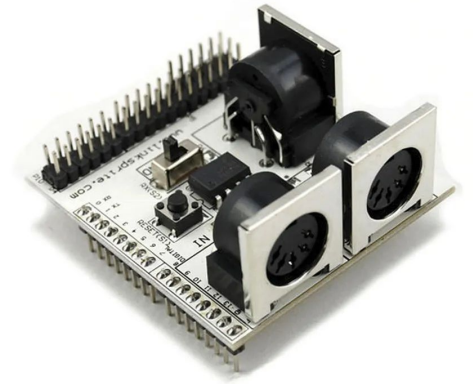
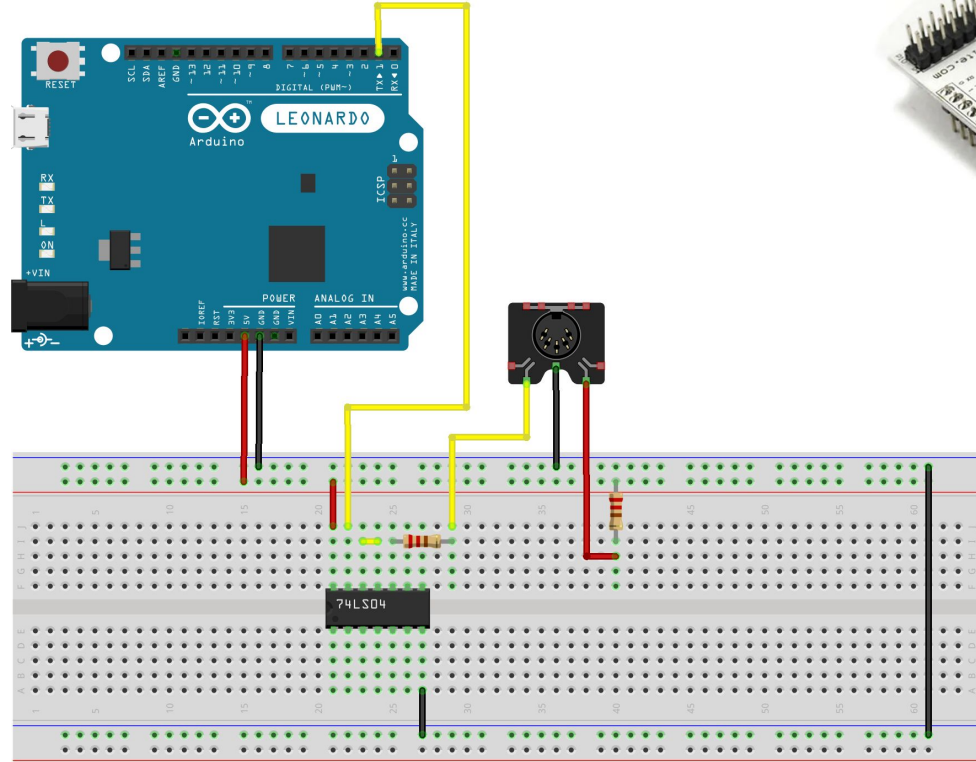
Sending Notes

Sending pitch bend

Basic MIDI sequencer

Fancy Sequencer

DIN MIDI out for Arduino



Basic Serial MIDI (demo)

```
const byte MIDI_CHANNEL = 1; // MIDI channels are 0-15 (0 = Channel 1)
const byte NOTE = 60; // Middle C
const byte VELOCITY = 100; // How hard the note is played
const int NOTE_DURATION = 500; // milliseconds

void setup() {
  Serial.begin(31250); // Start MIDI serial at standard baud rate
}

void loop() {
  // Send Note On: 0x90 + channel
  Serial.write(0x90 | MIDI_CHANNEL); // Note On message
  Serial.write(NOTE); // Note number
  Serial.write(VELOCITY); // Velocity

  delay(NOTE_DURATION); // Hold note

  // Send Note Off: either 0x80 or Note On with velocity 0
  Serial.write(0x90 | MIDI_CHANNEL); // Note On with velocity 0 (alternative Note Off)
  Serial.write(NOTE);
  Serial.write(0); // Velocity 0 = Note Off

  delay(1000); // Wait before repeating
}
```

Basic Serial MIDI with pitch bend (demo)

```
const byte MIDI_CHANNEL = 1;      // MIDI channel 1 (0-15)
const byte NOTE = 60;            // Middle C
const byte VELOCITY = 100;
const int STEP_DURATION = 30;     // Time between pitch bend steps
const int NOTE_DURATION = 2000;  // Total note time in ms

void setup() {
  Serial.begin(31250); // MIDI baud rate
}

void loop() {
  // Send Note On (status, note, velocity)
  Serial.write(0x90 | MIDI_CHANNEL); // 0x90 = Note On
  Serial.write(NOTE);
  Serial.write(VELOCITY);

  // Sweep Pitch Bend from center (8192) to max (16383)
  for (int value = 8192; value <= 16383; value += 256) {
    byte lsb = value & 0x7F; // lower 7 bits
    byte msb = (value >> 7) & 0x7F; // upper 7 bits

    Serial.write(0xE0 | MIDI_CHANNEL); // Pitch Bend message
    Serial.write(lsb);
    Serial.write(msb);

    delay(STEP_DURATION);
  }

  // Send Note Off (same status as Note On, velocity 0)
  Serial.write(0x90 | MIDI_CHANNEL);
  Serial.write(NOTE);
  Serial.write(0);

  delay(100); // Wait before repeating
}
```

Outline

What is MIDI?

5-Pin DIN MIDI

USB MIDI

Sending Notes

Sending pitch bend

Basic MIDI sequencer

Fancy Sequencer

Connect Arduino & Start IDE

Connect Arduino via USB

Start Arduino IDE



Select Arduino Leonardo



Install USB-MIDI Library

Sketch → Include Library → Manage Libraries...

Search for “**USB-MIDI**”

Install

1-Basic_IO

```
#include <USB-MIDI.h>

// Simple tutorial on how to receive and send MIDI messages.
// Here, when receiving any message on channel 4, the Arduino
// will blink a led and play back a note for 1 second.

USBMIDI_CREATE_DEFAULT_INSTANCE();

void setup()
{
    pinMode(LED_BUILTIN, OUTPUT);
    MIDI.begin();
}

void loop()
{
    if (true)
    {
        digitalWrite(LED_BUILTIN, HIGH);
        MIDI.sendNoteOn(42, 127, 1);    // Send a Note (pitch 42, velo 127 on channel 1)
        delay(1000);                    // Wait for a second
        MIDI.sendNoteOff(42, 0, 1);     // Stop the note
        digitalWrite(LED_BUILTIN, LOW);
        delay (100);
    }
}
```

Modifications

- Change the on and off times (delay time is milliseconds)
- Change the note
- Make it send a simple sequence of notes

Outline

What is MIDI?

5-Pin DIN MIDI

USB MIDI

Sending Notes

Sending pitch bend

Basic MIDI sequencer

Fancy Sequencer

Install Bounce2 Library

Sketch → Include Library → Manage Libraries...

Search for “**Bounce2**”

Install

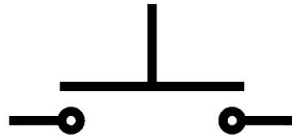
This handles the problem of switch “**bounce**” where pressing the button might trigger several close/open events due to mechanical bouncing.

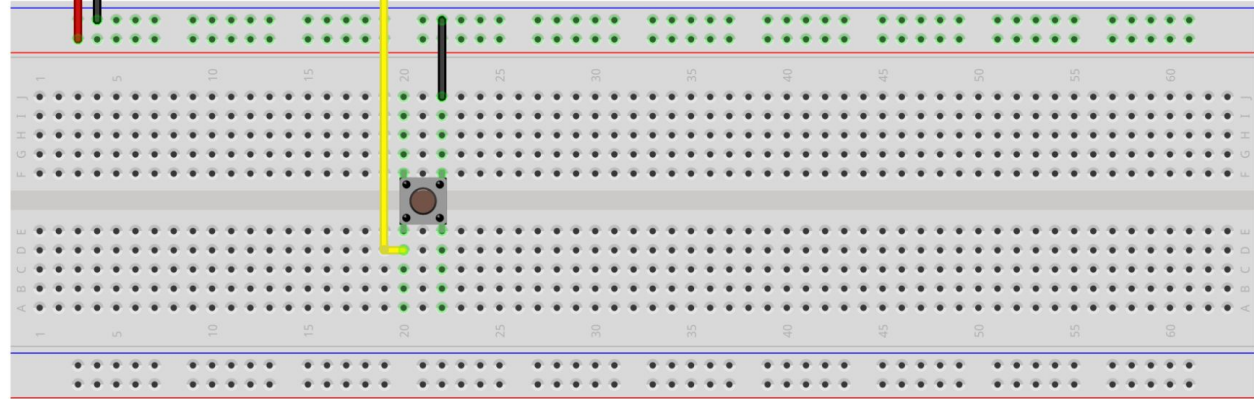
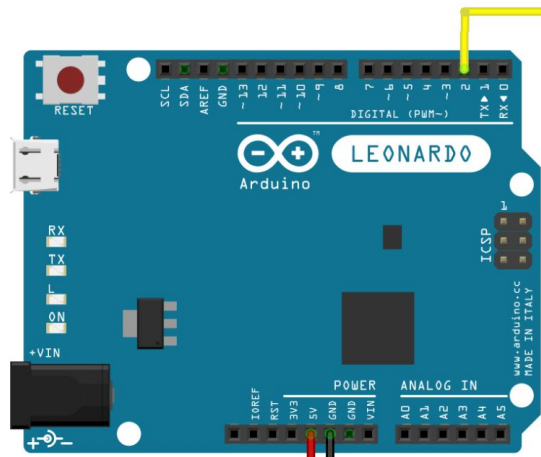
Debouncing works by ignoring rapid, repeated input signals for a short period after the first signal, ensuring only a single, clean input is registered.

Tactile switch (momentary)



pairs of pins already
connected - use opposite
corners if unsure





2-switch-note

```
#include <USB-MIDI.h>
#include <Bounce2.h>

// Create the default USB MIDI interface (named "MIDI")
USBMIDI_CREATE_DEFAULT_INSTANCE ();

const byte BUTTON_PIN = 2;
const byte NOTE = 60; // Middle C
const byte VELOCITY = 100;
const byte CHANNEL = 1;

Bounce button = Bounce ();

void setup () {
  pinMode (BUTTON_PIN, INPUT_PULLUP ); // Button connected to GND when pressed
  button.attach (BUTTON_PIN );
  button.interval (5); // Debounce interval in ms

  delay (1000);
  MIDI.begin (); // Start USB MIDI
}

void loop () {
  button.update ();

  if (button.fell ()) {
    // Button just pressed → send Note On
    MIDI.sendNoteOn (NOTE, VELOCITY, CHANNEL );
  }

  if (button.rose ()) {
    // Button just released → send Note Off
    MIDI.sendNoteOff (NOTE, 0, CHANNEL );
  }
}
```

Outline

What is MIDI?

5-Pin DIN MIDI

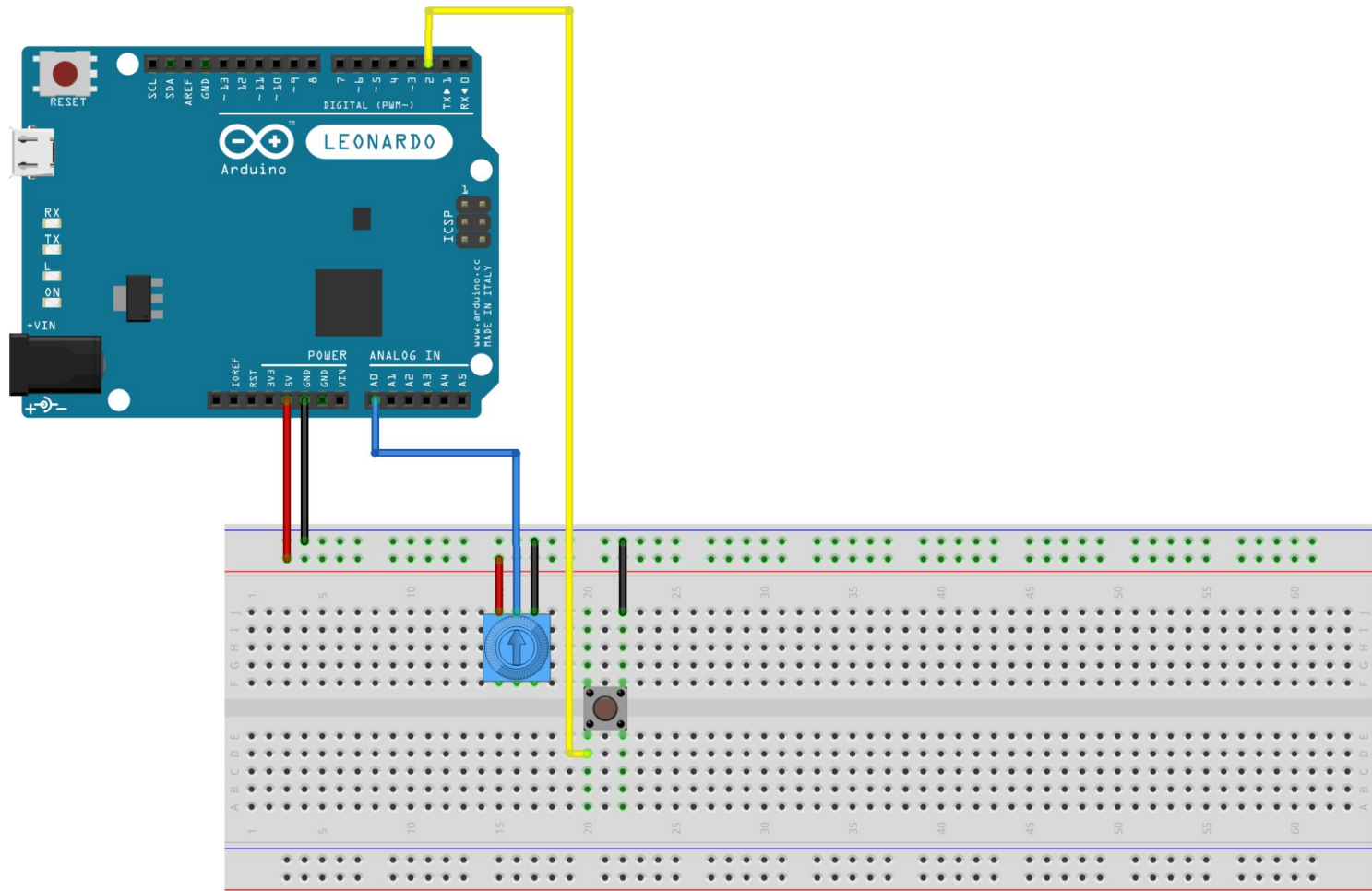
USB MIDI

Sending Notes

Sending pitch bend

Basic MIDI sequencer

Fancy Sequencer



3-pitchbend

```
#include <USB-MIDI.h>
#include <Bounce2.h>
#include <ResponsiveAnalogRead.h>

USBMIDI_CREATE_DEFAULT_INSTANCE();

const byte BUTTON_PIN = 2;
const byte NOTE = 60;
const byte VELOCITY = 100;
const byte CHANNEL = 1;
const byte POT_PIN = A0;

Bounce button = Bounce();
ResponsiveAnalogRead pot(POT_PIN, true); // use sleep mode = true

bool noteIsOn = false;

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  button.attach(BUTTON_PIN);
  button.interval(5);

  delay(1000); // Wait for USB MIDI host to stabilize
  MIDI.begin();
}
```

```
void loop() {
  button.update();
  pot.update(); // Filtered analog read

  // Handle note on/off from button
  if (button.fell()) {
    MIDI.sendNoteOn(NOTE, VELOCITY, CHANNEL);
    noteIsOn = true;
  }

  if (button.rose()) {
    MIDI.sendNoteOff(NOTE, 0, CHANNEL);
    noteIsOn = false;
  }

  // Only send pitch bend when the value changes
  if (pot.hasChanged()) {
    int analogValue = pot.getValue(); // 0-1023
    int bend = map(analogValue, 0, 1023, -8192, 8191);
    MIDI.sendPitchBend(bend, CHANNEL);
  }
}
```


Outline

What is MIDI?

5-Pin DIN MIDI

USB MIDI

Sending Notes

Sending pitch bend

Basic MIDI sequencer

Fancy Sequencer

Step Sequencer

We are now going to develop the sketch to play a **sequence** of 8 notes

The note sequence is stored in an **array**

An array is a list of values stored in order — a row of boxes, each holding a number

Each value has a position, called an index, starting from 0 — the first item is at index 0, the second at index 1, and so on

```
byte noteSequence[8] = {36, 0, 38, 43, 36, 41, 38, 0};  
// noteSequence[0] is 36  
// noteSequence[3] is 43 etc.
```

4-basic-sequencer

Listings are getting long... and you know what to do by now :-)

Sequencer controls:

Button: start/stop

Pot: Tempo

Modifications

- Change the sequence
- Change the BPM limits

Outline

What is MIDI?

5-Pin DIN MIDI

USB MIDI

Sending Notes

Sending pitch bend

Basic MIDI sequencer

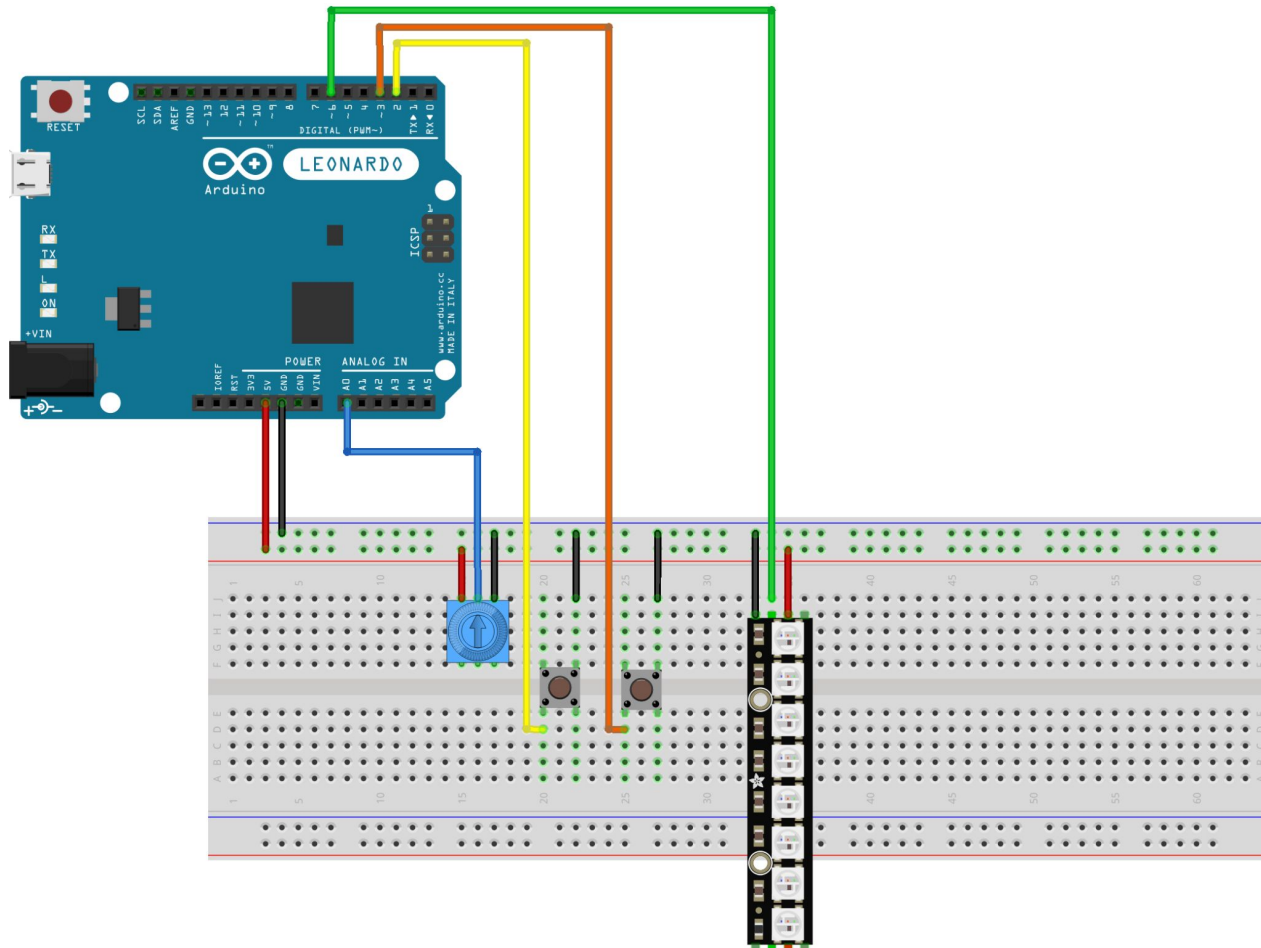
Fancy Sequencer

Fancy Step Sequencer

We are now going to add another button and the NeoPixel strip from Week 1

Expanding the sketch into a more capable sequencer:

- Edit notes
- Single stepping



Input Mode	
Button A	Long press: Switch to Play Mode
Button B	Short press: Enter a rest and advance step Long press: Preview note from pot, store on release
Pot	Select pitch for the current step
NeoPixel	Red = current step being edited Green = step contains note Off = rest

Play Mode	
Button A	Short press: Start/Stop playback Long press: Switch to Note Input Mode
Button B	Short press: Manually advance one step
Pot	Set tempo (BPM)
NeoPixel	Blue = current step playing Dim Blue = rest playing Green = note step

5-advanced-sequencer

Next Week...

Outline

What is Modular Synthesis?

Voltage controlled oscillators

Envelope generators

Voltage controlled filters

Low frequency oscillators

Modulation

Sound design for “drums”

Prerequisites

Week 1 [Tues, May 13th]

- Laptop with [Arduino IDE](#) installed
- Optional: [fritzing](#)
- USB-A to USB-C converter or hub if needed
- Arduino starter kit (provided)



Week 2 [Tues, May 20th]

- Laptop with [Ableton Live](#) and [MIDIView](#) installed
- Alternatively [Garageband](#), [Tracktion](#) or whatever DAW
- Headphones
- Arduino starter kit



Week 3 [Tues, May 27th]

- Laptop with [VCV Rack](#) Installed
- Headphones



Week 4 [Tues, June 3rd]

- Laptop with Arduino IDE, Ableton Live, VCV Rack and MIDIView installed
- Headphones
- Arduino starter kit